PostPPP Java SDK

集成开发指南

SDK 1.0.0 | PostPPP Go 原生库 1.0 | 2026-10-01

文档说明

本指南面向 postppp-java-lib 集成人员，说明 Java/JNA 调用方式、PostpppInfo 原生结构映射、JSON 返回对象、字段含义、错误处理、动态库和测试基线。SDK 封装 postppp-go-lib 的 C ABI，一次同步调用完成一个测站一天的静态精密单点定位（PPP）解算。

文档基线：Java SDK 1.0.0，原生版本接口当前返回 1.0。字段顺序、数组长度、字符串编码和返回 JSON 字段名属于接口契约；升级原生库时应同时核对匹配的头文件、动态库和本指南。

免责声明

本指南按当前源码与已归档测试证据编写，不构成对特定业务适用性、结果精度或持续兼容性的额外保证。调用方应自行核验许可证、观测数据、广播星历、精密产品、天线文件、时间窗口和运行平台。

文档修改记录

版本	修改日期	描述
v1.0.0	2026-10-01	补充 PostpppInfo 完整字段映射、返回 JSON 的 Java 对象封装、中文字段解释、安装与故障排查。

一 SDK 简介

PostPPP Java SDK 是 postppp-go-lib 原生 PPP 算法库的 Java/JNA 封装。服务层提供结构体入口和 JSON 入口，并把 native 返回的 UTF-8 JSON 解析为可直接使用的 Java 对象。

层级	入口	适用场景	主要职责
业务服务层	PostpppLibService	Spring Boot 或普通 Java 应用	版本、许可证、结构体/JSON 解算及 typed response。
结果解析层	JsonUtils	已有 JSON 或自定义协议处理	校验响应、保留原文、解析 PostpppResponse/PostpppResult。
原生绑定层	PostpppLibrary	高级集成或底层调试	通过 JNA 加载 libPostppp.so 并负责 Pointer 生命周期。
输入结构层	PostpppInfo	结构体解算	按 Go PostpppInput 的 ABI 顺序写入 native 内存。

二 名词解释

名词	说明
PPP	精密单点定位；本 SDK 面向单测站、单日静态解算。
JNA	Java Native Access，用于加载动态库并映射 C 结构体与函数。
GPST	GPS 时间；PostpppTime 以 GPS 周和周内秒表示。
ECEF / ENU / LLH	地心地固坐标、东-北-天局部坐标、大地坐标。
OSB/BIA / ANTEX	码偏差文件和天线校准文件；由原生库读取。

数据对象与单位

名词	说明
PostpppInput	Go C ABI 的输入结构体；Java 对应 PostpppInfo，大小为 392 字节。
Result	Go 返回 JSON 中的单测站单日 PPP 结果；Java 对应 PostpppResult。
Response	Go 返回 JSON 的 code、error、result 外层；Java 对应 PostpppResponse。
历元	一个观测时刻的解算结果，对应 PostpppEpoch；时间对应 PostpppTime。
固定宽度字段	C char[] 字段，必须使用 byte[] 并按 UTF-8 字节容量写入。
部分结果	code=-6 时 native 仍返回 result；用于诊断，不表示成功。

接口契约中的单位

数据	单位与顺序
ECEF 坐标	X/Y/Z，米。
ENU 偏差	东/北/天，米。
LLH	纬度/经度为度，大地高为米。
时间	GPS 周 + 周内秒（GPST）。
对流层	ZTD/ZHD/ZWD、梯度和标准差，米。

三 快速开始

1. 准备工作

准备 JDK 20 或以上、Maven 3.x、有效许可证，以及与解算日期匹配的观测文件、广播星历、精密轨道、钟差、码偏差和天线文件。原生库不下载这些数据。

2. 支持平台

运行平台	JAR 资源目录	状态
Linux x86-64	linux-x86-64/libPostppp.so	当前验证平台；已归档 12 项测试全部通过。
Linux AArch64	linux-aarch64/libPostppp.so	资源目录存在；需用同架构 native 构建验证。
macOS Apple Silicon	darwin-aarch64/libPostppp.so	当前资源文件实际为 Linux ELF，不能直接作为 macOS 库使用。
Windows	无	工程未提供。

3. 安装 SDK

```
mvn clean verify
mvn install
```

Maven 坐标为 com.navfirst:postppp-java-lib:1.0.0。产物是普通依赖 JAR：target/postppp-java-lib-1.0.0.jar；Spring Boot Maven 插件跳过可执行 JAR 重打包。

```
<dependency>
  <groupId>com.navfirst</groupId>
  <artifactId>postppp-java-lib</artifactId>
  <version>1.0.0</version>
</dependency>
```

4. 接入流程

按以下顺序接入：确认 Linux amd64 与 native 库版本，申请有效许可证，准备单测站单日观测和同日期的广播/精密产品，构建输入，调用同步解算，检查 code，再读取质量统计和坐标。需要输出文件时提前创建可写目录并设置对应开关。

5. 获取算法版本和机器标识

```
System.out.println(sdk.getVersion());
String uuid = sdk.startUuid();
```

getVersion 返回 native 算法版本（本轮为 1.0）；startUuid 返回许可证绑定所需的机器标识。不要把 native 版本 1.0 与 Maven SDK 版本 1.0.0 混用。

6. 构建输入并启动解算

```
PostpppLibService sdk = new PostpppLibServiceImpl();
PostpppInfo input = PostpppInfo.builder()
        .year(2026).month(7).day(1)
        .bia("/data/bias.BIA")
        .atx("/data/igs20.atx")
        .obsFile("/data/ABCD1820.26o")
        .build();
input.setNavPaths("/data/BRDC.rnx");
input.setSp3Paths("/data/orbit.SP3");
input.setClkPaths("/data/clock.CLK");
input.setStationNameText("ABCD");


PostpppResponse response = sdk.solvePostpppResponse(input, "/data/license.lic");
if (!response.isSuccess()) {
    throw new IllegalStateException(response.getError());
}
PostpppResult result = response.getResult();
System.out.println(result.getFinalLlh()[0]);
```

new PostpppInfo()、PostpppInfo.defaults() 和 builder().build() 使用与 Go 初始化器一致的默认值。手动调用 initPostpppInput 会覆盖已填写字段，应在填写输入前调用。

四 详细接口描述

1. 版本与许可证

接口	参数	返回值与说明
getVersion()	无	返回当前加载的原生 PPP SDK 版本字符串。
startUuid()	无	返回机器标识，通常用于申请设备绑定许可证。
setLicense(String)	许可证路径	校验并缓存许可证；成功返回 JSON，失败返回空字符串。
getLicense(String)	许可证路径	读取许可证元数据但不更新缓存；失败时可能返回普通错误文本。

1.1 接口定义

```
String getVersion();
String startUuid();
String setLicense(String path);
String getLicense(String path);
```

1.2 参数与返回值

许可证路径可传绝对路径。setLicense 成功后会在当前 native 进程缓存许可证；getLicense 只读取元数据，不改变缓存。许可证 JSON 中只有 active=true 才表示有效。

2. 解算入口

接口	用途	失败行为
solvePostppp(PostpppInfo, String)	调用 solvePostpppStruct 结构体入口。	参数为空抛 IllegalArgumentException；native 错误仍以 JSON 返回。
solvePostppp(String, String)	调用 solvePostppp JSON 入口。	JSON 为空、含 NUL、超 64 MiB 或非法代理字符时由 JsonUtils 校验。
solvePostpppResponse(...)	返回 PostpppResponse，保留 code/error/可选 result。	适合需要处理错误码和 -6 部分结果的业务。
solvePostpppResult(...)	直接返回成功的 PostpppResult。	native code 非 0 时抛 PostpppException。

2.1 接口定义

```
String solvePostppp(PostpppInfo input, String license);
String solvePostppp(String inputJson, String license);
PostpppResponse solvePostpppResponse(...);
PostpppResult solvePostpppResult(...);
```

2.2 参数说明

结构体入口中的 license 为空时使用同一进程先前成功设置的许可证缓存；非空时按该路径校验。JSON 入口使用 Go 字段名，数组 nav/sp3/clk 最多 4 项。

2.3 返回值说明

solvePostppp 原始入口返回完整 JSON 字符串；typed response 解析外层状态并保留部分结果；typed result 只接受 code=0，非零 code 转换为 PostpppException。

3. JSON 请求示例

```
String request = """
   {"year":2026,"month":7,"day":1,
    "nav":["/data/BRDC.rnx"],
    "sp3":["/data/orbit.SP3"],
    "clk":["/data/clock.CLK"],
    "bia":"/data/bias.BIA",
    "atx":"/data/igs20.atx",
    "obs_file":"/data/ABCD1820.26o",
    "station_name":"ABCD"}
   """;
PostpppResponse response = sdk.solvePostpppResponse(request, "/data/license.lic");
```

JSON 请求使用 Go 协议字段名，例如 elev_mask、obs_file、station_name；JsonUtils.toJson 只序列化传入对象，不自动把 Java 驼峰名称转换为 Go 字段名。

4. 低层 JNA 接口

native 函数	Java 声明	说明
getPVersion	Pointer getPVersion()	返回 native 版本指针；服务层读取并释放。
getUuid	Pointer getUuid()	返回机器 UUID 指针。
setLicense / getLicense	Pointer ...(String path)	读取许可证信息；setLicense 会更新 native 缓存。
solvePostppp	Pointer solvePostppp(String, String)	JSON 入口；返回 UTF-8 JSON 指针。
solvePostpppStruct	Pointer solvePostpppStruct(PostpppInfo, String)	结构体入口；返回 UTF-8 JSON 指针。
freeP	void freeP(Pointer)	只释放 native 导出的返回字符串。

五 数据模型

1. PostpppInfo 输入模型

PostpppInfo 对应 Go main.go 中的 PostpppInput，而不是底层 postppp_opt_t。当前 64 位 ABI 大小为 392 字节，字段顺序必须保持一致。C int32_t 映射为 Java int，C double 映射为 Java double，C char[] 映射为 byte[]。

Java 字段	C / JSON 字段	类型	中文说明与默认值
year / month / day	year / month / day	int	GPST 日期；全 0 从观测首历元确定，年范围 1970～2099。
interval	interval	double	采样间隔（秒）；0 自动推断，不能为负。
navsys	navsys	int	卫星系统位掩码；默认 13，即 GPS+GAL+BDS。
elevMask	elev_mask	double	截止高度角（度）；默认 7，范围 [0,90)。
soltype	soltype	int	解算方向；0 前向、1 反向、2 双向组合（默认）。
tropopt	tropopt	int	对流层模型；3 仅 ZTD，4 为 ZTD+梯度（默认）。
bdsfreq	bdsfreq	int	BDS 频点组合；4=B1I+B3I（默认），5=B1C+B2A。
tidecorr	tidecorr	int	潮汐改正；0 关闭、1 固体潮（默认）、2 全潮汐。
tropPrn	trop_prn	double	对流层过程噪声；0 使用内核默认，不能为负。
maxSigma	max_sigma	double	三维位置中误差门限（米）；默认 0.020，0 关闭。
enuOffset	enu_offset	double[3]	ENU 平移（米），默认全 0；顺序东/北/天。
outDir	out_dir	String	输出目录；null/空值时不写文件。
writePos / writeTro / writeQc	write_pos / write_tro / write_qc	int	非 0 分别写 POS、SINEX_TRO、质量报告。
troInterval	tro_interval	double	SINEX_TRO 间隔（秒）；默认 300，0 也回退到 300。
software / agency	software / agency	String	结果元数据；为空时 native 回退默认名称。
verbose	verbose	int	日志级别；当前 C ABI 无日志回调，单独设置不会输出 Java 日志。

路径、字符数组和数组字段

Java 字段	C / JSON 字段	类型	中文说明与默认值
nav	nav	Pointer[4]	广播星历路径，最多 4 项；成功读到一项即止。
sp3 / clk	sp3 / clk	Pointer[4]	精密轨道和钟差路径，各最多 4 项；多文件可累加读取。
bia / atx	bia / atx	String	必需的 OSB/BIA 码偏差与 ANTEX 天线文件。
erp / blq / ionex	erp / blq / ionex	String	ERP/BLQ 为全潮汐 tidecorr=2 时的必需文件；IONEX 为可选电离层格网。
obsFile	obs_file	String	必需的 RINEX 或 RTCM3 观测文件路径。
stationName	station_name	byte[16]	测站名；最多 15 个 UTF-8 字节加 NUL。
approxXyz	approx_xyz	double[3]	ECEF 先验坐标（米）；全 0 时使用观测头坐标。
antType	ant_type	byte[24]	天线类型；最多 23 个 UTF-8 字节加 NUL。
radome	radome	byte[8]	天线罩名称；最多 7 个 UTF-8 字节加 NUL。
antDelta	ant_delta	double[3]	天线 E/N/U 偏心（米），默认全 0。

nav/sp3/clk 使用 Pointer[4] 是因为 JNA 不支持在 Structure 中直接写入 String[]。setNavPaths、setSp3Paths、setClkPaths 会分配 UTF-8、NUL 结尾的 Memory，并保留引用直到结构体使用完毕；空项写为 NULL。定长 char[] 不能映射为 Java char[]。

```
input.setStationNameText("ABCD");
input.setAntTypeText("TRM59800.00");
input.setRadomeText("NONE");
input.setNavPaths("/data/BRDC.rnx");
input.setSp3Paths("/data/orbit.SP3", "/data/orbit-next.SP3");
input.setClkPaths("/data/clock.CLK");
```

五 数据模型（续）

2. 返回对象

native 返回 JSON 的外层通过 PostpppResponse 封装；成功时 result 为 PostpppResult，-6（没有合格 PPP 解）时也可能保留部分 result，便于诊断。字段使用 Gson 的 @SerializedName 保持 Go 下划线协议，Java getter 使用驼峰命名。

1. PostpppResponse

JSON 字段	Java 属性 / getter	类型	中文说明
code	code / getCode()	int	原生返回码；0 成功，非 0 表示参数、文件或解算错误。
error	error / getError()	String	错误描述；成功时通常为空。
result	result / getResult()	PostpppResult	PPP 结果；成功时必有，-6 时可能是部分结果。
—	isSuccess()	boolean	等价于 code == 0。
—	hasResult()	boolean	result 是否存在。
—	isPartialResult()	boolean	code == -6 且 result 存在时为 true。

2. PostpppTime 与 PostpppEpoch

JSON 字段	Java getter	类型	中文说明
week / tow	getWeek() / getTow()	int / double	GPS 周和周内秒（GPST）。
time	getTime()	PostpppTime	历元时间。
xyz / sigma	getXyz() / getSigma()	double[3]	ECEF 坐标及标准差，顺序 X/Y/Z，单位米。
sigma3d	getSigma3d()	double	ECEF 三维位置标准差，单位米。
enu	getEnu()	double[3]	相对参考坐标的 ENU 偏差，顺序东/北/天，单位米。
ztd / zhd / zwd	getZtd() / getZhd() / getZwd()	double	总、干、湿对流层延迟，单位米。
gn / ge	getGn() / getGe()	double	北向/东向对流层梯度，单位米。
sd_ztd / sd_gn / sd_ge	getSdZtd() / getSdGn() / getSdGe()	double	对应参数标准差，单位米。
nsat	getNsat()	int	参与该历元 PPP 解算的卫星数。

JSON 字段	Java getter	类型	中文说明
stat	getStat()	int	0 无解、3 SPP、4 PPP 浮点、6 PPP 固定。
accepted	isAccepted()	boolean	是否通过位置质量门限并计入最终统计。

3. PostpppResult（测站日结果）

JSON 字段	Java getter	类型	中文说明
name / year / doy	getName() / getYear() / getDoy()	String / int	测站名、年份、年积日。
epoch	getEpoch()	List<PostpppEpoch>	按 GPST 排序的逐历元结果；拒绝历元也保留。
have_coord / n_coord	isHaveCoord() / getNCoord()	boolean / int	是否有合格坐标及合格历元数。
first / last	getFirst() / getLast()	PostpppTime	有效成果起止时间。
final_xyz / final_sigma	getFinalXyz() / getFinalSigma()	double[3]	最后合格历元的 ECEF 坐标及标准差，单位米。
final_llh	getFinalLlh()	double[3]	最后合格历元大地坐标，纬度/经度为度，高为米。
mean_xyz	getMeanXyz()	double[3]	合格历元 ECEF 平均值，单位米。
rms_enu	getRmsEnu()	double[3]	相对合格均值的 ENU 重复性 RMS，单位米。
bias_enu / bias_enu_mean	getBiasEnu() / getBiasEnuMean()	double[3]	最后历元/全天平均 ENU 偏差，顺序东/北/天。
ztd_mean	getZtdMean()	double	全天合格成果平均 ZTD，单位米。
n_epoch / n_ppp / n_accept	getNEpoch() / getNPpp() / getNAccept()	int	总历元、PPP 历元、质量门限通过历元数量。
n_gated / n_spp / n_none	getNGated() / getNSpp() / getNNone()	int	被门限剔除、仅 SPP、无结果的历元数量。
avail / mean_nsat	getAvail() / getMeanNsat()	double	数据可用率（百分比）及平均可用卫星数。
n_gap / max_gap_sec / max_gap_start	getNGap() / getMaxGapSec() / getMaxGapStart()	int / double / PostpppTime	中断窗口数、最长中断秒数及开始时间。
converge_sec	getConvergeSec()	double	首次达到收敛条件的耗时，单位秒。
grade / software / agency	getGrade() / getSoftware() / getAgency()	String	综合质量等级及结果元数据。
products	getProducts()	String	使用的精密产品标识。

JSON 字段	Java getter	类型	中文说明
pos_file / tro_file	getPosFile() / getTroFile()	String	POS、SINEX_TRO 实际输出路径；未输出时为空。

4. typed API 示例

```
PostpppResponse response = sdk.solvePostpppResponse(input, license);
if (!response.isSuccess()) {
    System.err.printf("PPP failed (%d): %s%n",
            response.getCode(), response.getError());
}
if (response.getResult() != null) {
    PostpppResult result = response.getResult();
    System.out.println(result.getName());
    System.out.println(result.getFinalXyz()[0]);
    System.out.println(result.getEpoch().size());
}
```

六 状态码

1. 返回码

code	含义	处理建议
0	成功	读取 result；仍应检查 have_coord、grade 和质量统计。
-1	参数或许可证错误	检查 JSON、日期、路径、许可证和输入文件。
-2	内存不足	减少输入规模或并发数，检查 native 进程资源。
-3	广播星历读取错误	核对 nav 路径、日期覆盖和文件格式。
-4	精密产品读取错误	核对 SP3/CLK/BIA/ATX 与日期匹配。
-5	观测文件读取错误	核对 obs_file、权限和 RINEX/RTCM3 格式。
-6	没有合格 PPP 解	优先使用 PostpppResponse 保留部分 result 做诊断。
-7	输出文件错误	检查 outDir 权限、磁盘空间和输出开关。

2. 历元解状态与质量标志

字段与值	含义	业务判断
stat=0 / 3	无解 / SPP	不能作为合格 PPP 成果。
stat=4 / 6	PPP 浮点解 / PPP 固定解	结合 accepted 和质量统计判断。
accepted=false	该历元未通过位置质量门限	保留用于诊断，不参与合格坐标汇总。
have_coord=false	没有合格坐标成果	不能解释 final_xyz/final_llh 中的零值。
grade	native 综合质量等级	用于质量汇总，不能替代与独立真值的精度检验。

七 使用限制说明

1. 输入与 native 限制

- year/month/day 全 0 时由首历元确定；指定日期时必须同时填写有效年月日。
- nav、sp3、clk 每类最多 4 个路径；路径字符串必须可按 UTF-8 转换且不能包含 NUL。
- station_name、ant_type、radome 受固定字节容量限制，中文字符可能占用多个 UTF-8 字节。
- tidecorr=2 时需要 ERP 和 BLQ；缺少精密产品、BIA 或 ANTEX 会导致 native 错误。
- outDir 只有和 writePos、writeTro 或 writeQc 同时设置时才会生成对应输出。
- PostpppInfo 是 native 内存映射对象，调用期间不要修改字段、数组或路径 Memory。

2. Spring Boot 配置

```
ppp.enabled=true


# 关闭默认自动配置
ppp.enabled=false
```

默认自动配置通过 PostpppAutoConfiguration 注册 PostpppLibService。提供自定义 PostpppLibService Bean 时，@ConditionalOnMissingBean 会保留自定义实现。

3. native 内存生命周期

服务层会把 native 返回的 char* 复制为 UTF-8 Java String，并在 finally 中调用一次 freeP。直接使用 PostpppLibrary 时返回值是 Pointer，调用方必须读取并释放，不能重复释放；输入结构体及其路径 Memory 不应传给 freeP。

4. 同步与并发

服务调用是同步阻塞的。调用期间不要修改 PostpppInfo 及其数组。当前 Go 内核存在进程级锁；多站并行处理应在验证 native 线程安全后再部署，保守方案是不同进程或在业务层串行保护。

5. Spring Boot 与动态库

Spring Boot 3 应用可以注入 PostpppLibService；普通 Java 程序直接 new PostpppLibServiceImpl。JNA 以 Postppp 名称加载动态库，外部库可以通过 -Djna.library.path=/path/to/native 指定。加载失败会抛出 UnsatisfiedLinkError。

```
java -Djna.library.path=/opt/postppp      -jar your-app.jar
```

6. 输出目录与覆盖策略

outDir 应按站点、日期和任务实例分配唯一目录。writePos、writeTro、writeQc 由业务明确开启，重复任务不要共享同一目录，避免 POS、SINEX_TRO 和质量文件互相覆盖。

7. 输入文件与许可证

许可证、观测文件、广播星历、SP3、CLK、BIA、ANTEX、ERP 和 BLQ 必须覆盖同一解算日期和测站。SDK 只负责传递路径，不负责下载、校验和归档这些外部文件。

八 常见问题

1. 启动时报 UnsatisfiedLinkError

- 确认操作系统和 CPU 架构与资源目录一致；当前 Linux x86-64 是已验证平台。
- 确认 JAR 中包含 linux-x86-64/libPostppp.so，或通过 -Djna.library.path 指定外部目录。
- 确认 native 动态库与 PostpppInfo 字段顺序来自同一 Go 构建。

2. 返回 code=-6

code=-6 表示没有合格 PPP 解，但 native 可能同时提供部分 result。需要诊断时使用 solvePostpppResponse 并检查 haveCoord、nAccept、grade、epoch 和 error；只处理成功分支时才使用 solvePostpppResult。

3. JSON 解析失败

- 响应必须是对象且 code 必须为数字；成功 code=0 时必须存在 result 对象。
- error 必须是字符串；result 必须是对象或 null。
- 请求 JSON 不能为空、不能包含 NUL 或未配对代理字符，大小上限为 64 MiB。

4. 坐标结果为零或输出文件为空

先判断 result.haveCoord；为 false 时不要解释坐标数组中的零值。需要落盘时同时设置 outDir 和对应 write 开关，并检查路径权限、磁盘空间和文件覆盖策略。

九 测试、迁移与复现

1. 当前测试基线

当前归档基线为 Linux amd64 Docker 中的 12 项自动化测试，覆盖 PostpppInfo 的 ABI 大小、字段默认值、UTF-8 定长字段、路径数组、JSON 响应对象和 native 错误路径。测试报告与逐项 Surefire 证据位于 docs/test-evidence/2026-10-01。

检查项	当前结果	集成含义
ABI 与默认值	392 字节；字段顺序、数组长度和默认值断言通过	升级 Go 结构体时必须同步更新 Java 映射。
typed JSON	Response、Result、Epoch、Time 均可解码	业务层可按 getter 读取中文字段对应的结果属性。
错误路径	code/error、-6 部分结果和异常 facade 通过	生产代码应优先检查 code，再读取 result。
native 冒烟	版本、许可证错误、结构体和 JSON 入口通过	证明动态库可加载和 C ABI 可调用，不等于精度验收。

2. 版本迁移提示

- 升级 postppp-go-lib 时先比较 PostpppInput 字段顺序、sizeof、定长 char 数组和返回 JSON 字段，再更新 Java 映射。
- 如果新增 JSON 字段，优先在 domains 包增加属性和中文说明；未知字段由 Gson 忽略，不应依赖未知字段完成业务判断。
- 如果返回码含义变化，应同时更新状态码表、PostpppException、测试夹具和测试报告。
- 发布新 native 文件后重新执行 Linux amd64、目标架构和许可证错误冒烟，并重新归档 SHA-256。

3. 复现与证据

```
mvn -B -o clean verify
python3 scripts/report-tests/run_report_tests.py
python3 scripts/docs/build_docs.py
```

完整字段与 Go 端结果定义见 postppp-go-lib/readme.md；测试报告、Markdown 版报告和原始 Surefire 证据见 docs/。

附录 A 完整 Spring Boot 示例

```
@SpringBootApplication
public class PppApplication {
    public static void main(String[] args) {
        SpringApplication.run(PppApplication.class, args);
    }
}

@Service
class PppJob {
    private final PostpppLibService sdk;

    PppJob(PostpppLibService sdk) { this.sdk = sdk; }

    PostpppResult solve(PostpppInfo input, String license) {
        return sdk.solvePostpppResult(input, license);
    }
}
```

默认情况下 ppp.enabled=true，自动配置会注册 PostpppLibService；设置 ppp.enabled=false 后可完全关闭默认 Bean。

附录 B 直接 JNA 调用示例

```
PostpppInfo input = PostpppInfo.defaults();
input.setNavPaths("/data/BRDC.rnx");
input.bia = "/data/bias.BIA";
input.atx = "/data/igs20.atx";
input.obsFile = "/data/ABCD.26o";
input.write();

Pointer pointer = PostpppLibrary.INSTANCE.solvePostpppStruct(input, license);
try {
    String json = pointer.getString(0, "UTF-8");
} finally {
    PostpppLibrary.INSTANCE.freeP(pointer);
}
```

低层调用必须自己负责 Pointer 的读取与释放；业务代码优先使用 PostpppLibService，避免把 native 生命周期泄漏到应用层。

附录 C 工程包结构

路径	职责
library/PostpppInfo.java	PostpppInput JNA 结构、默认值、UTF-8 路径和定长字段。
library/PostpppLibrary.java	C ABI 函数声明、同步 JNA 实例和 freeP。
services/PostpppLibService.java	高层同步服务和 typed API 默认方法。
services/impl/PostpppLibServiceImpl.java	读取并释放 native 返回字符串。
domains/*.java	响应、结果、历元和时间 Java 对象。
utils/JsonUtils.java	请求验证、响应校验和 Gson 解码。
configurations/*	Spring Boot 自动配置和 ppp.enabled 属性。

附录 D 构建与验证

```
mvn -B -o clean verify
python3 scripts/report-tests/run_report_tests.py
python3 scripts/docs/build_docs.py
```

发布前同时检查目标平台 native 文件、PostpppInfo.size()、测试报告中的 code/error 断言和 docs 下的最终 PDF。